ЛАБОРАТОРНА РОБОТА

Архітектурний опис системи за допомогою UML діаграм

Частина 1 – текст для файлу README.md

Система: Система бронювання квитків.

Виконав: Агаєв Дамір Миколайович, група КН-4.

1. Опис предметної області

Система бронювання квитків дозволяє користувачам онлайн переглядати заходи (концерти, вистави, кіносеанси), обирати місця та оформлювати замовлення. Користувач реєструється в системі, обирає подію та квитки на вільні місця й створює замовлення, яке може містити один або кілька квитків. Замовлення оплачується через платіжний сервіс, після успішної оплати користувач отримує електронний квиток. Система повинна контролювати доступність місць, щоб один квиток не було продано двічі, та дозволяти скасування замовлення і повернення квитка.

2. Архітектурний підхід та Component Diagram

Для системи обрано шарову (Layered) архітектуру в моделі клієнт-сервер: клієнтська частина (веб- або мобільний застосунок) звертається до серверної через HTTPS у форматі JSON, а серверна частина поділена на шари Presentation, Business Logic і Data Access. Кожен шар залежить лише від шару, розташованого нижче, що розділяє відповідальність і дозволяє змінювати окремі частини (інтерфейс, платіжний провайдер, СУБД) без переробки решти системи.

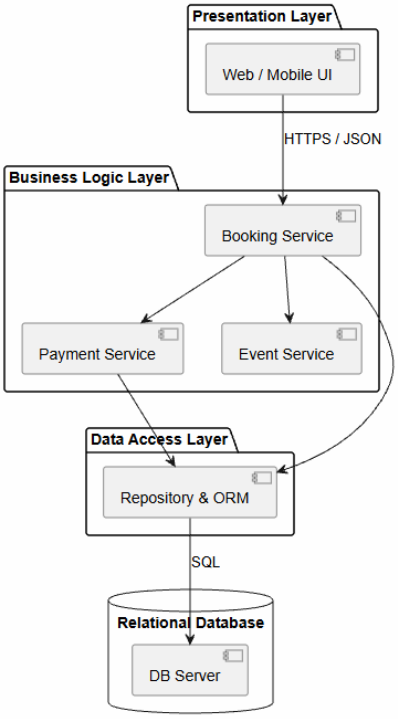


Рис. 1. UML Component Diagram

Шар	Компонент	Роль
Presentation Layer	Web / Mobile UI	Відображення подій і місць, форми замовлення, прийом дій користувача; передає запити серверу, не містить бізнес-правил.
Business Logic Layer	Booking Service	Керує створенням і скасуванням замовлення, координує роботу інших сервісів.
	Payment Service	Виконує оплату замовлення та повертає результат.
	Event Service	Надає інформацію про події та наявні місця.
Data Access Layer	Repository & ORM	Відображає об'єкти на таблиці, виконує читання й запис даних, приховує SQL від бізнес-логіки.
Relational Database	DB Server	Постійне зберігання даних користувачів, подій, квитків, замовлень і платежів.

Взаємодія між шарами (за діаграмою). Web / Mobile UI надсилає запити до Booking Service через HTTPS / JSON. Booking Service звертається до Payment Service для оплати та до Event Service для даних про події, а для збереження даних використовує Repository & ORM. Payment Service також зберігає результати платежів через Repository & ORM. Repository & ORM виконує SQL-запити до DB Server. Залежності спрямовані зверху вниз, тому інтерфейс не працює з даними безпосередньо.

3. UML Class Diagram

Діаграма містить п'ять сутностей: User, Event, Ticket, Booking, Payment.

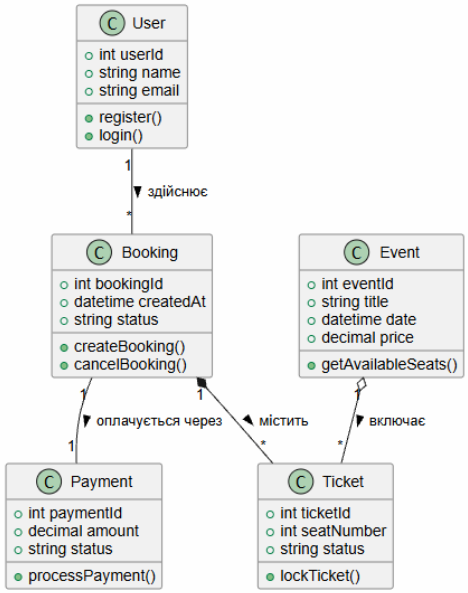


Рис. 2. UML Class Diagram

Клас	Призначення	Атрибути	Методи
User	Користувач системи	userId (int), name (string), email (string)	register(), login()
Event	Подія, на яку продаються квитки	eventId (int), title (string), date (datetime), price (decimal)	getAvailableSeats()
Ticket	Квиток на конкретне місце	ticketId (int), seatNumber (int), status (string)	lockTicket()
Booking	Замовлення користувача	bookingId (int), createdAt (datetime), status (string)	createBooking(), cancelBooking()
Payment	Оплата замовлення	paymentId (int), amount (decimal), status (string)	processPayment()

Зв'язки між класами:

Зв'язок	Тип	Кратність	Пояснення
User – Booking («здійснює»)	Асоціація	1 : *	Користувач може здійснити багато замовлень; кожне замовлення належить одному користувачу.
Booking – Payment («оплачується через»)	Асоціація	1 : 1	Кожне замовлення оплачується одним платежем.
Booking – Ticket («містить»)	Композиція	1 : *	Квитки входять до складу замовлення як його невід'ємні частини: життєвий цикл квитка в замовленні залежить від замовлення.
Event – Ticket («включає»)	Агрегація	1 : *	Подія об'єднує набір квитків, але квиток є окремою сутністю й не прив'язаний жорстко до життєвого циклу події.

Наслідування в моделі не використовується: у предметній області немає ієрархії типів, усі п'ять класів є самостійними сутностями.

4. Функціональні сценарії: Use Case та Sequence Diagrams

Ключова функція системи – **«Створити замовлення квитка»**. Основний актор – Користувач; зовнішній актор – Платіжний шлюз, який виконує оплату. Прецедент

«Створити замовлення квитка» включає (include) перегляд доступних місць та оплату замовлення.

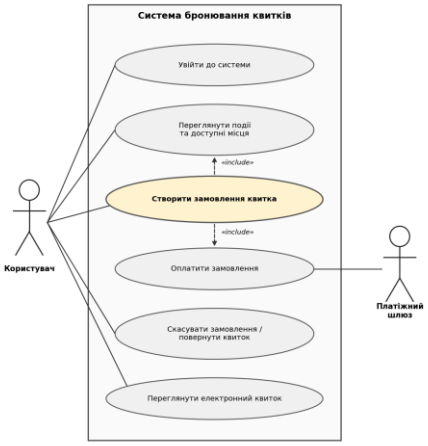


Рис. 3. UML Use Case Diagram

Прецедент	Опис
Увійти до системи	Автентифікація користувача (методи register() та login()).
Переглянути події та доступні місця	Користувач переглядає список подій і вільні місця (getAvailableSeats()).
Створити замовлення квитка	Оформлення замовлення на вибрані квитки; включає перегляд місць та оплату.
Оплатити замовлення	Виконується через зовнішній платіжний шлюз.
Скасувати замовлення / повернути квиток	Скасування замовлення користувачем (cancelBooking()).
Переглянути електронний квиток	Відображення квитка після успішної оплати.

Сценарій «Створити замовлення квитка» докладно показано на діаграмі послідовності.

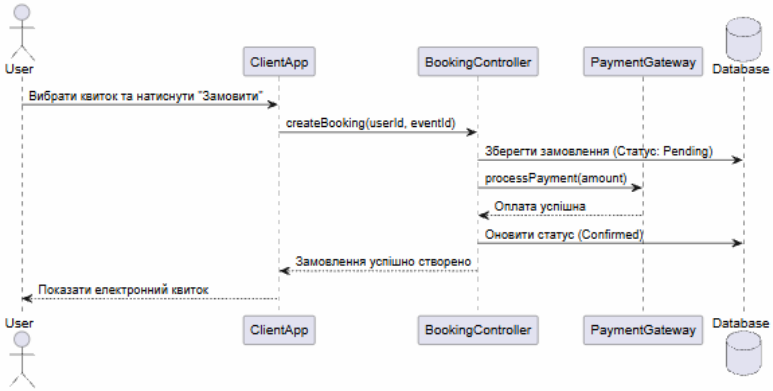


Рис. 4. UML Sequence Diagram: створення замовлення

Учасники: User, ClientApp, BookingController, PaymentGateway, Database. Послідовність кроків (успішний сценарій):

1. Користувач (User) обирає квиток у ClientApp і натискає «Замовити».

2. ClientApp викликає метод createBooking(userId, eventId) у BookingController.
3. BookingController зберігає замовлення в Database зі статусом Pending.
4. BookingController викликає processPayment(amount) у PaymentGateway.
5. PaymentGateway повертає повідомлення «Оплата успішна».
6. BookingController оновлює статус замовлення в Database на Confirmed.
7. BookingController повідомляє ClientApp, що замовлення успішно створено.
8. ClientApp показує користувачу електронний квиток.

Статус Pending у цьому сценарії відповідає стану PendingPayment на діаграмі станів. Помилка або тайм-аут оплати переводять замовлення в стан Cancelled (розділ 5). Відповідність компонентів: ClientApp – Web / Mobile UI, BookingController – Booking Service, PaymentGateway – Payment Service та зовнішній платіжний провайдер.

5. State Machine Diagram

Життєвий цикл сутності **Booking (Замовлення)** показано на діаграмі станів. Початковий перехід виконується з початкового псевдостану, кінцевий стан досягається після завершення життєвого циклу.

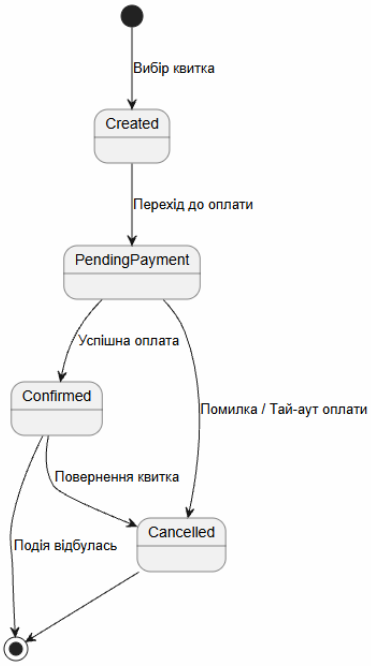


Рис. 5. UML State Machine Diagram: замовлення (Booking)

Стан	Значення
Created	Замовлення створено після вибору квитка; оплата ще не розпочата.
PendingPayment	Замовлення очікує оплати.
Confirmed	Оплату підтверджено, квиток дійсний.
Cancelled	Замовлення скасовано через помилку або тайм-аут оплати чи повернення квитка.

Переходи між станами:

З стану	У стан	Подія
Початковий стан	Created	Вибір квитка
Created	PendingPayment	Перехід до оплати
PendingPayment	Confirmed	Успішна оплата
PendingPayment	Cancelled	Помилка / тайм-аут оплати
Confirmed	Cancelled	Повернення квитка
Confirmed	Кінцевий стан	Подія відбулась
Cancelled	Кінцевий стан	Завершення життєвого циклу замовлення

Архітектурний опис системи «Система бронювання квитків»

Частина 2 – документ для збереження у форматі PDF

Виконав: Агаєв Дамір Миколайович, група КН-4.

1. Мета проєкту

Розробити архітектуру невеликої системи онлайн-бронювання квитків на події, яка дозволяє переглядати події, обирати місця, створювати та оплачувати замовлення й отримувати електронний квиток, а також описати цю архітектуру за допомогою UML діаграм.

2. Тип архітектури

Шарова (Layered) архітектура в моделі клієнт-сервер. Клієнт (Web / Mobile UI) взаємодіє із сервером через HTTPS / JSON. Серверна частина поділена на три шари: Presentation, Business Logic, Data Access; дані зберігаються в реляційній базі даних. Підхід обрано, тому що він розділяє відповідальність між шарами, спрощує тестування та дозволяє змінювати окремі частини (інтерфейс, платіжний провайдер, СУБД) без переробки решти системи.

3. Основні компоненти

- **Presentation Layer:** Web / Mobile UI – інтерфейс користувача.
- **Business Logic Layer:** Booking Service (замовлення), Payment Service (оплата), Event Service (події та місця).
- **Data Access Layer:** Repository & ORM – доступ до даних.
- **Relational Database:** DB Server – зберігання даних.

4. UML діаграми

- **Component Diagram** – шари та залежності між компонентами (рис. 1).
- **Class Diagram** – п'ять сутностей User, Event, Ticket, Booking, Payment та зв'язки між ними: асоціація, агрегація, композиція (рис. 2).
- **Use Case Diagram** – актори та прецеденти ключової функції «Створити замовлення квитка» (рис. 3).
- **Sequence Diagram** – послідовність взаємодії об'єктів під час створення замовлення (рис. 4).
- **State Machine Diagram** – життєвий цикл замовлення: Created → PendingPayment → Confirmed / Cancelled (рис. 5).

5. Короткі пояснення та обґрунтування

Користувач через інтерфейс створює замовлення; Booking Service координує перевірку доступності місць, оплату через Payment Service та збереження даних через

Repository & ORM. Замовлення проходить стани від створення до підтвердження або скасування, що відображено на діаграмі станів. Діаграми узгоджені між собою: сутності класів відповідають даним сервісів, а сценарій послідовності відповідає компонентам архітектури.

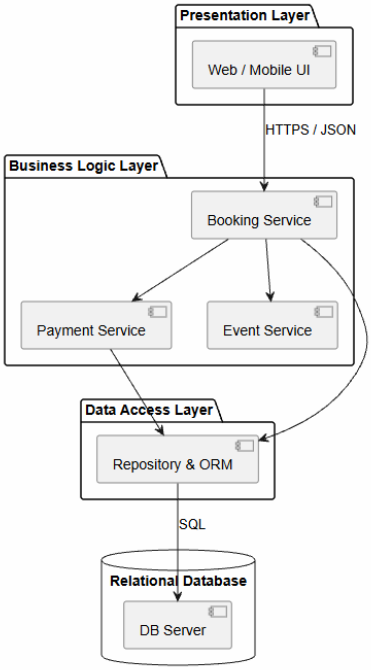


Рис. 1. Component Diagram

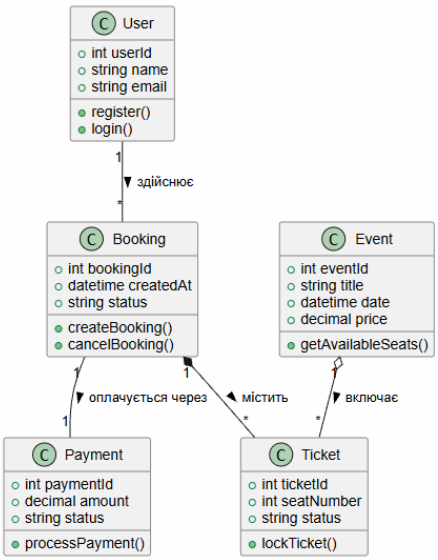


Рис. 2. Class Diagram

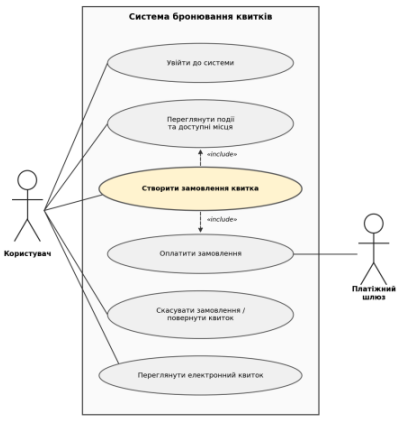


Рис. 3. Use Case Diagram

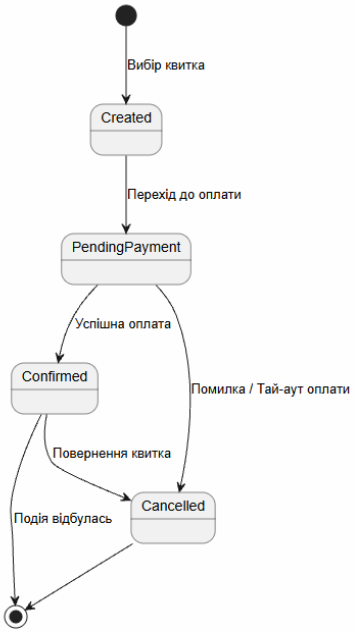


Рис. 5. State Machine Diagram

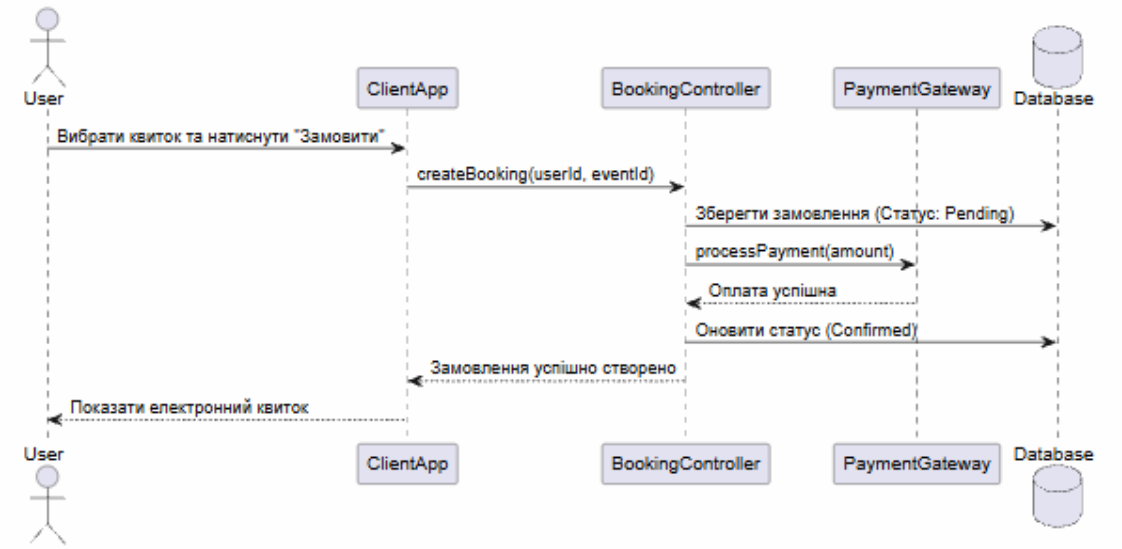


Рис. 4. Sequence Diagram: створення замовлення